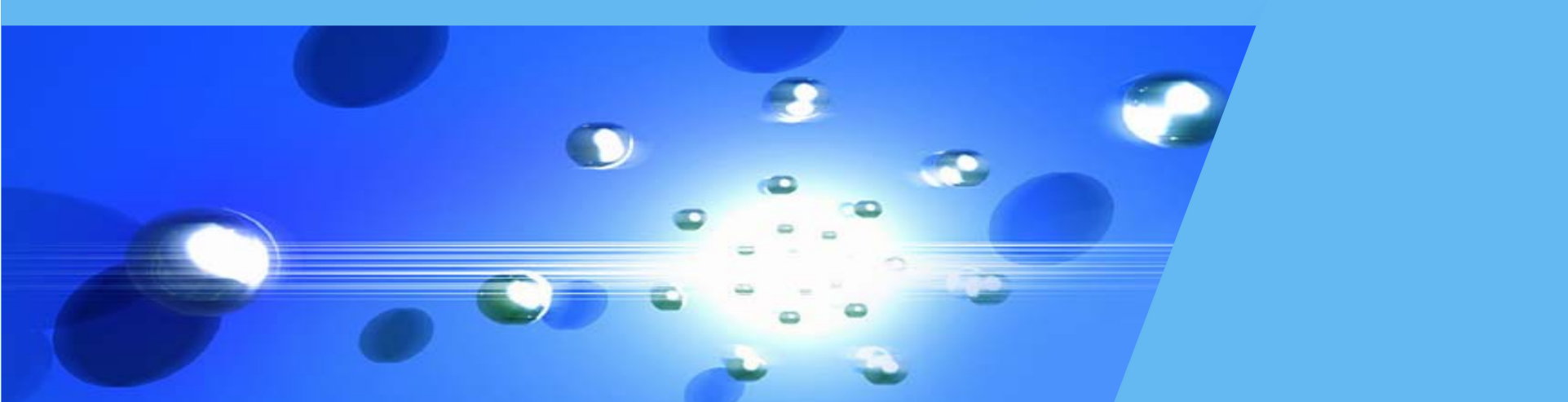


软件项目实训



第五章 Vue.js基础

石 雷

合肥工业大学计算机与信息学院

课程提纲

1

Web前端开发概述

2

Vue.js开发基础

3

计算属性与侦听器

4

事件处理

5

表单绑定

6

结构渲染

7

阶段案例

一. Web前端开发概述

❖ Web开发简史



- **早期阶段:** 1995年以前, 仅仅是“内容开发”, 有简单的CGI, 可动态生成HTML。
- **服务器端模板时代:** 1995-2005年, 以ASP/JSP/PHP为代表, 有业务逻辑代码, 但完全嵌入在页面 (Page) 中。
- **服务器MVC时代:** 2006-2015年, 以Java SSH、ASP.NET MVC等各种框架为代表, 出现了各种基于MVC模式的后端框架, 开发强调分层。有前后端概念, 但并未分离。
- **前后端分离时代:** 2012年开始, 2016年后爆发。前后端分离的开发模式成为主流。

一. Web前端开发概述

❖ 基于前后端分离模式的Web开发

（一）从提供内容到提供服务的转变

- 传统互联网3个特点：使用场景固定且局限、“内容”为主、“服务”局限于特定领域；
- 移动互联网3个特点：使用场景触达社会每个角落、更多事物被连接到云端、海量“服务”；
- 对技术上的3个要求：
 - 客户端需求复杂化，大量应用流行，对用户体验的期望提高；
 - 客户端渲染成为“刚需”；
 - 客户端程序不得不具备完整的生命周期、分层架构和技术栈。

一. Web前端开发概述

❖ 基于前后端分离模式的Web开发

(二) 从“单一网站”到“多终端应用”

- 服务器端通过API输出数据，剥离“视图”；
- Web客户端变成独立开发和部署的程序，不再是服务器端Web程序中的“前端”层；
- 每个客户端都倾向于拥有专门为自己量身打造、可被自己掌控的API网站。

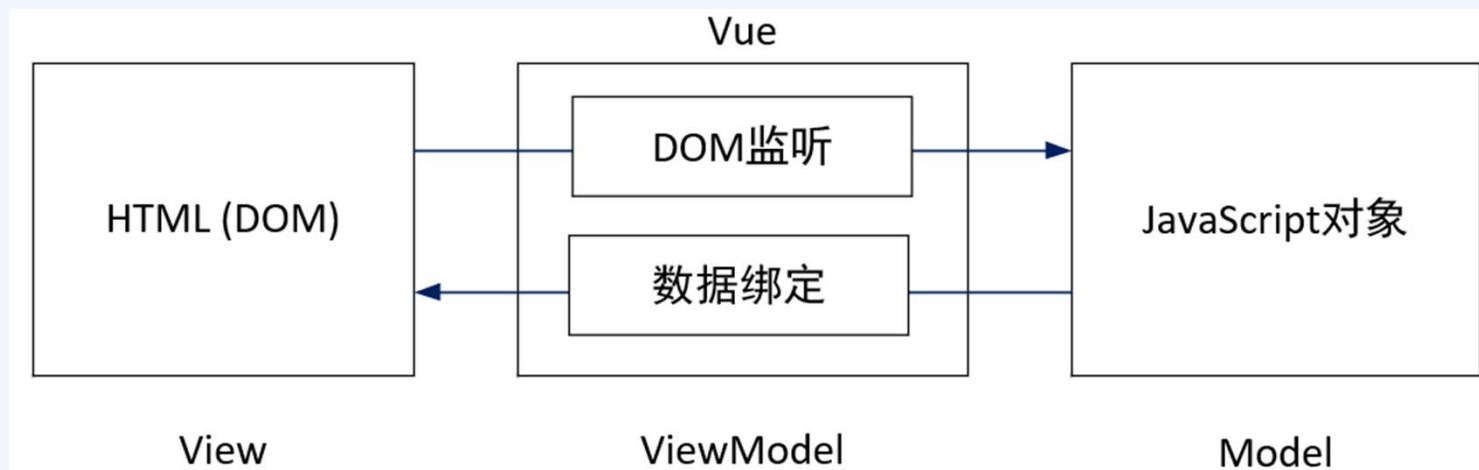
在移动时代，一个应用往往需要适配不同的终端形态

- 桌面应用：传统的Windows应用、Mac应用；
- 移动应用：iOS、安卓应用；
- Web：通过浏览器访问的应用；
- 超级APP：以微信小程序为代表的超级APP，成为新的应用程序平台。

一. Web前端开发概述

❖ Vue.js与MVVM模式

Vue.js是2014年诞生的、专门针对前后端分离开发模式的、用于构建用户界面的**渐进式框架**。具有轻量级、双向数据绑定、组件化管理、插件化开发等特征。



- **数据双向绑定**，View和Model之间不直接沟通，而是通过ViewModel这个桥梁进行交互；
- 使用“**声明式**”的编程理念。

一. Web前端开发概述

❖ 安装Vue.js

简单方法：简单通过<script>标记引入，适用于简单页面开发；

完整方法：使用相关的命令行工具进行完整安装，适用于组件化的项目开发。

```
<script src="https://cdn.jsdelivr.net/npm/vue@2.7.10/dist/vue.js"></script>
```

一. Web前端开发概述

❖ 上手实践：第一个Vue.js程序

猜数游戏

请猜一个1~100之间的整数

55

太小了，往大一点猜

87

祝贺你，你猜对了

课程提纲

1

Web前端开发概述

2

Vue.js开发基础

3

计算属性与侦听器

4

事件处理

5

表单绑定

6

结构渲染

7

阶段案例

二. Vue.js开发基础

❖ Vue根实例

Vue.js的核心工作就是创建一个“ViewModel”对象，并将它作为“视图”和“模型”的桥梁。具体语法形式如下：（变量类型，变量声明）

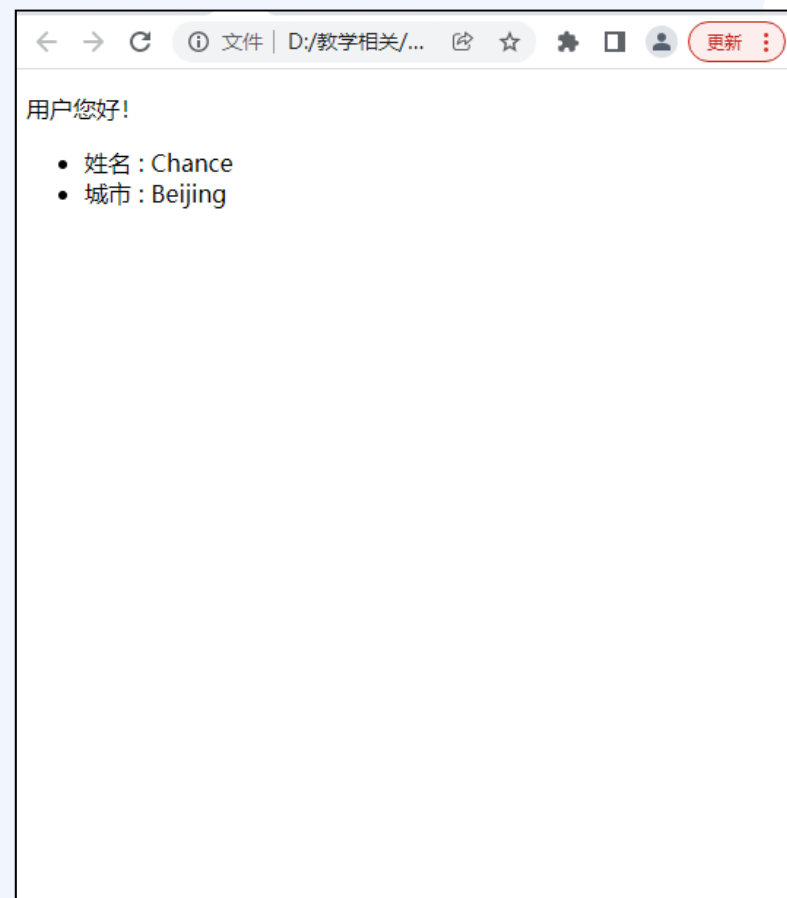
```
let vm = new Vue({  
    // 选项对象  
})
```

1. var定义的变量，没有块的概念，可以跨块访问，不能跨函数访问。
2. let定义的变量，只能在块作用域里访问，不能跨块访问，也不能跨函数访问。
3. const定义的变量，使用时必须初始化（即必须赋值），只能在跨作用域里访问，而且不能修改。

二. Vue.js开发基础

❖ Vue根实例——文本差值

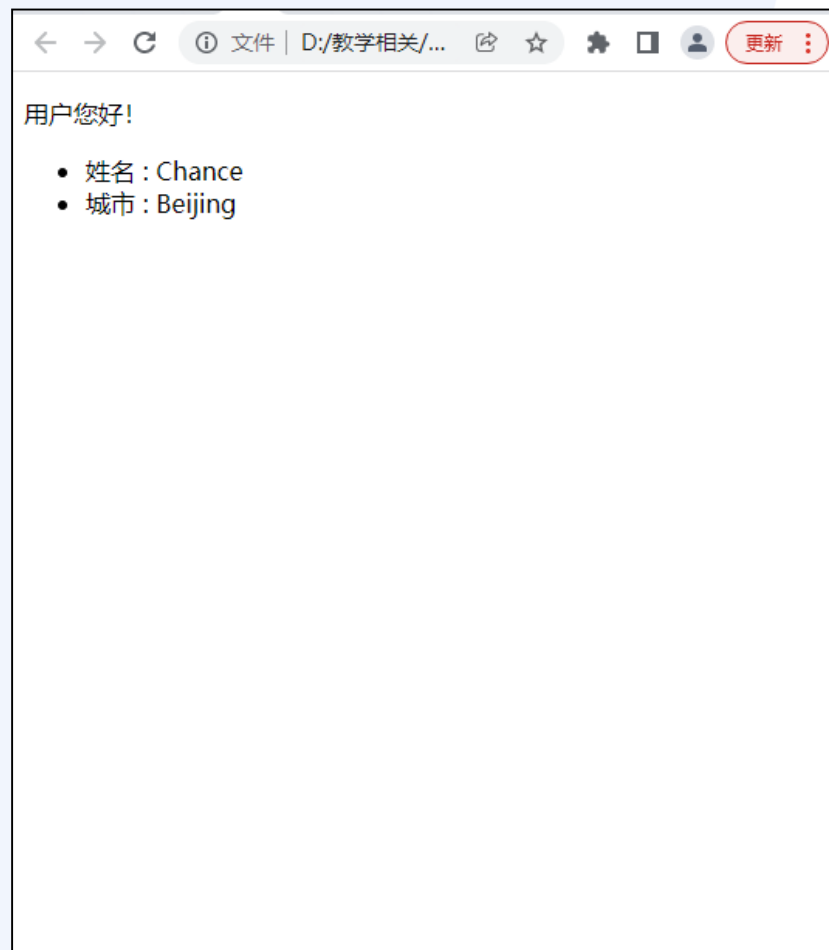
```
instance-01.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>用户信息-文本插值-data返回对象</title>
  <script src="vue.js"></script>
</head>
<body>
  <div id="app">
    <p>用户您好! </p>
    <ul>
      <li>姓名 : {{name}}</li>
      <li>城市 : {{city}}</li>
    </ul>
  </div>
  <script>
    let user = {
      name : "Chance",
      city : "Beijing",
    };
    let vm = new Vue({
      el: '#app',
      data: user
    })
  </script>
</body>
</html>
```



二. Vue.js开发基础

❖ Vue根实例——文本差值

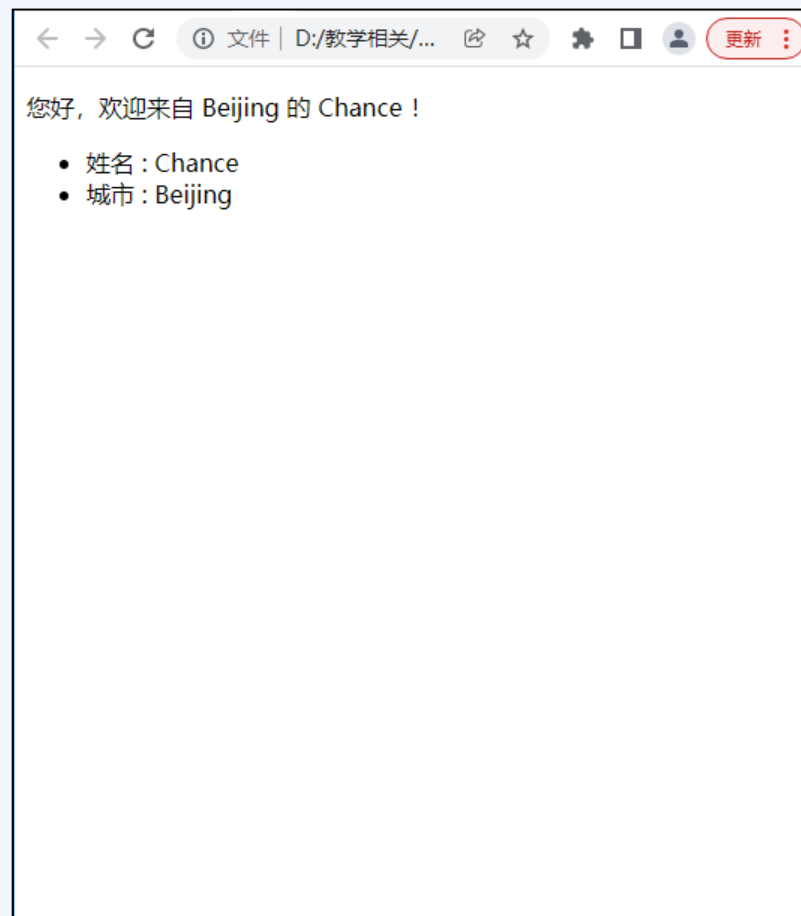
```
instance-02.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>用户信息-文本插值-data返回函数</title>
  <script src="vue.js"></script>
</head>
<body>
  <div id="app">
    <p>用户您好! </p>
    <ul>
      <li>姓名 : {{name}}</li>
      <li>城市 : {{city}}</li>
    </ul>
  </div>
  <script>
    let user = {
      name : "Chance",
      city : "Beijing",
    };
    let vm = new Vue({
      el: '#app',
      data() {
        return user;
      }
    })
  </script>
</body>
</html>
```



二. Vue.js开发基础

❖ Vue根实例——方法属性

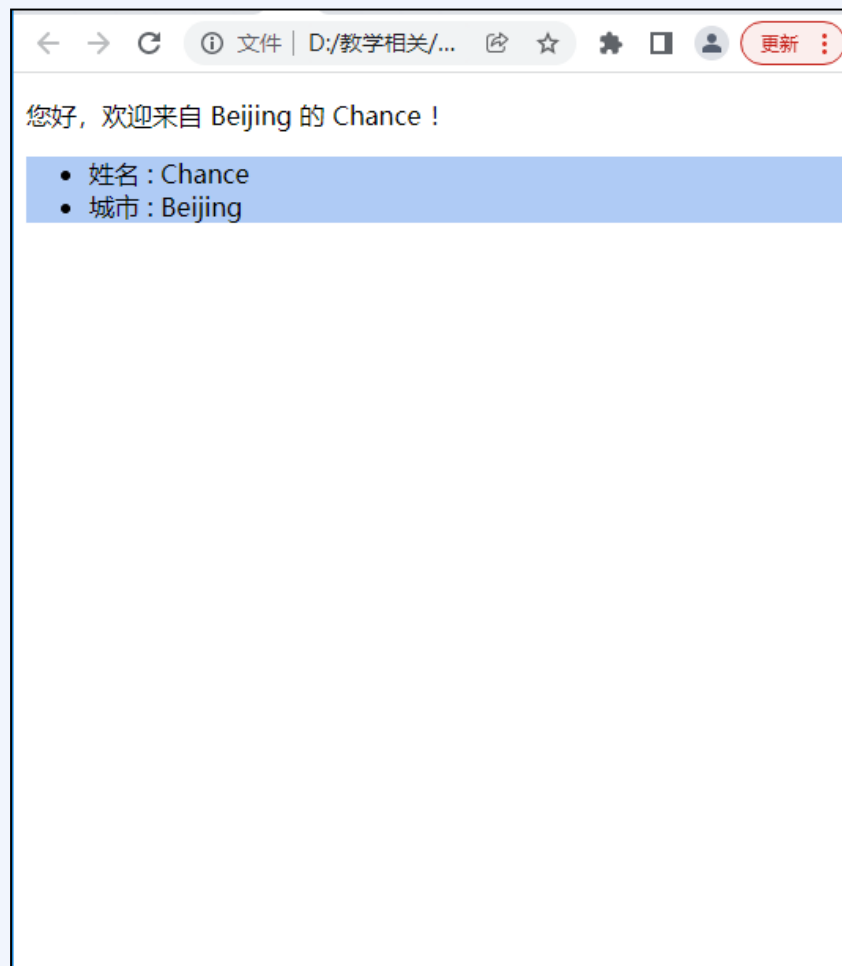
```
instance-04.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>用户信息-方法属性</title>
  <script src="vue.js"></script>
</head>
<body>
  <div id="app">
    <p>{{sayHello()}}</p>
    <ul>
      <li>姓名 : {{name}}</li>
      <li>城市 : {{city}}</li>
    </ul>
  </div>
  <script>
    let user = {
      name : "Chance",
      city : "Beijing"
    };
    let vm = new Vue({
      el: '#app',
      data: user,
      methods: {
        sayHello() {
          return `您好, 欢迎来自 ${this.city} 的 ${this.name} !`
        }
      }
    })
  </script>
</body>
</html>
```



二. Vue.js开发基础

❖ Vue根实例——属性绑定

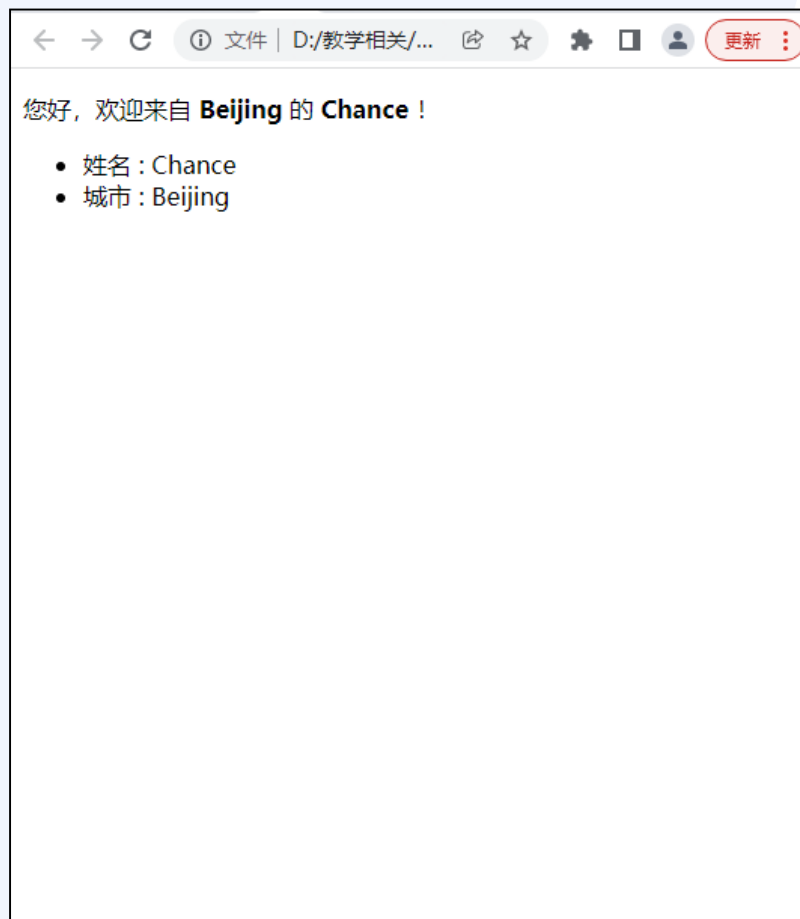
```
instance-06.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>用户信息-属性绑定-绑定class属性</title>
  <script src="vue.js"></script>
  <style>
    .male{
      background-color: rgb(175, 203, 245);
    }
    .female{
      background-color: rgb(248, 213, 241);
    }
  </style>
</head>
<body>
  <div id="app">
    <p>{{sayHello()}}</p>
    <ul v-bind:class="sex">
      <li>姓名 : {{name}}</li>
      <li>城市 : {{city}}</li>
    </ul>
  </div>
</body>
<script>
  let user = {
    name: "Chance",
    city: "Beijing",
    sex: "male"
  };
  let vm = new Vue({
    el: '#app',
    data: user,
    methods: {
      sayHello() {
        return `您好, 欢迎来自 ${this.city} 的 ${this.name} !`
      }
    }
  })
</script>
</body>
</html>
```



二. Vue.js开发基础

❖ Vue根实例——插入HTML片段

```
instance-07.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>用户信息-插入html片段</title>
  <script src="vue.js"></script>
</head>
<body>
  <div id="app">
    <p v-html="sayHello()"></p>
    <ul>
      <li>姓名 : {{name}}</li>
      <li>城市 : {{city}}</li>
    </ul>
  </div>
  <script>
    let user = {
      name : "Chance",
      city : "Beijing"
    };
    let vm = new Vue({
      el: '#app',
      data: user,
      methods: {
        sayHello() {
          return `您好, 欢迎来自 <b>${this.city}</b> 的 <b>
${this.name}</b> !`
        }
      }
    })
  </script>
</body>
</html>
```



二. Vue.js开发基础

❖ Vue中的指令

■ 什么是指令?

指令 (Directives) 是带有 “v-” 前缀的特殊属性。指令属性的值预期是单JavaScript表达式 (v-for是例外情况)。

■ 有哪些常见指令?

v-text、**v-html**、**v-model**、**v-cloak**、**v-bind (:)**、**v-on (@)**、**v-if**、**v-show**、**v-for**、**v-once**、**v-pre**

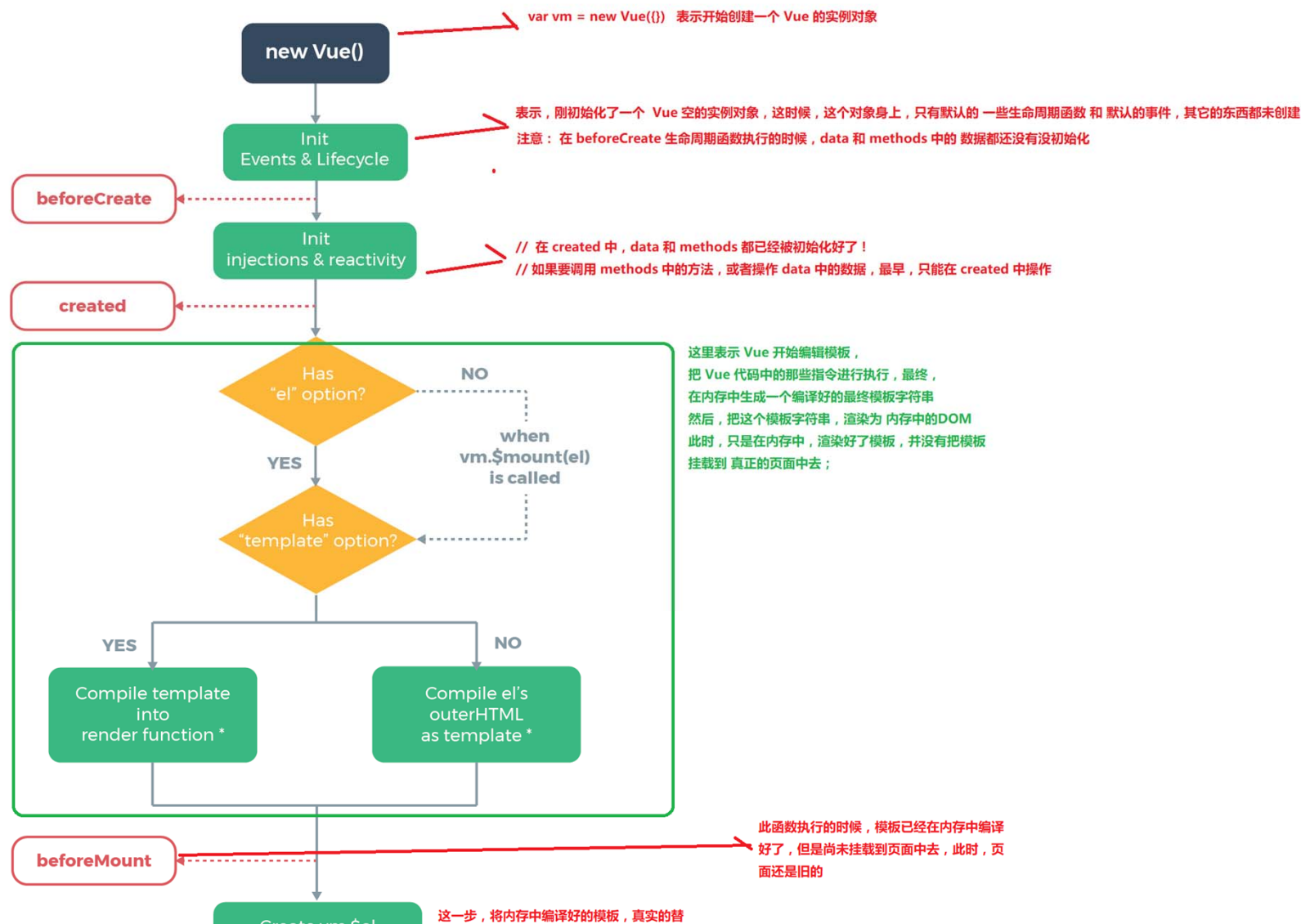
二. Vue.js开发基础

❖ Vue实例的生命周期 <https://cn.vuejs.org/>

Vue生命周期函数就是Vue实例在某一个时间点会自动执行的函数

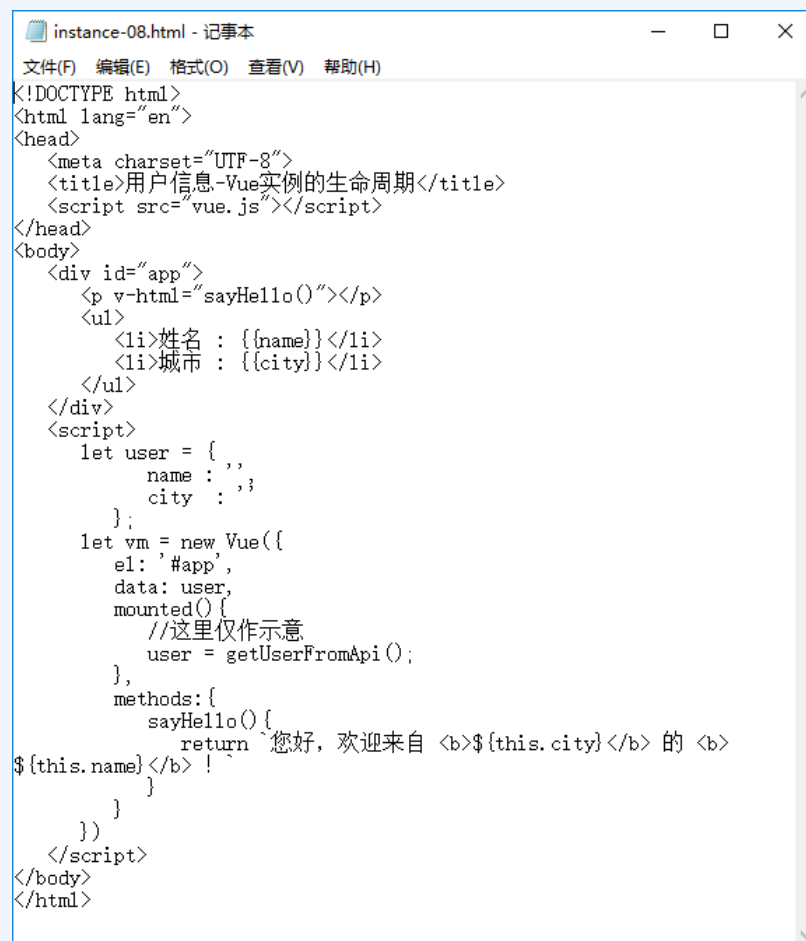
- 当Vue对象**创建之前**触发的函数（beforeCreate）
- 当Vue对象**创建完成**触发的函数（Created）
- 当Vue对象**开始挂载数据**的时候触发的函数（beforeMount）
- 当Vue对象**挂载数据完成**的时候触发的函数（Mounted）
- Vue对象中的**data数据发生改变之前**触发的函数（beforeUpdate）
- Vue对象中的**data数据发生改变完成**触发的函数（Updated）
- Vue对象**销毁之前**触发的函数（beforeDestroy）
- Vue对象**销毁完成**完成的函数（Destroy）

二. Vue.js开发基础



二. Vue.js开发基础

❖ Vue实例的生命周期



```
instance-08.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>用户信息-Vue实例的生命周期</title>
  <script src="vue.js"></script>
</head>
<body>
  <div id="app">
    <p v-html="sayHello()"></p>
    <ul>
      <li>姓名 : {{name}}</li>
      <li>城市 : {{city}}</li>
    </ul>
  </div>
  <script>
    let user = {
      name : '',
      city : ''
    };
    let vm = new Vue({
      el: '#app',
      data: user,
      mounted() {
        //这里仅作示意
        user = getUserFromApi();
      },
      methods: {
        sayHello() {
          return `您好, 欢迎来自 <b>${this.city}</b> 的 <b>
${this.name}</b> !`
        }
      }
    })
  </script>
</body>
</html>
```

课程提纲

1

Web前端开发概述

2

Vue.js开发基础

3

计算属性与侦听器

4

事件处理

5

表单绑定

6

结构渲染

7

阶段案例

三. 计算属性与侦听器

❖ 计算属性与侦听器

一个应用程序中，变量之间存在很多关联关系。Vue.js提供了根据有些变量自动关联另一些变量的机制，以简化对象之间的复杂关系。

❖ 计算属性

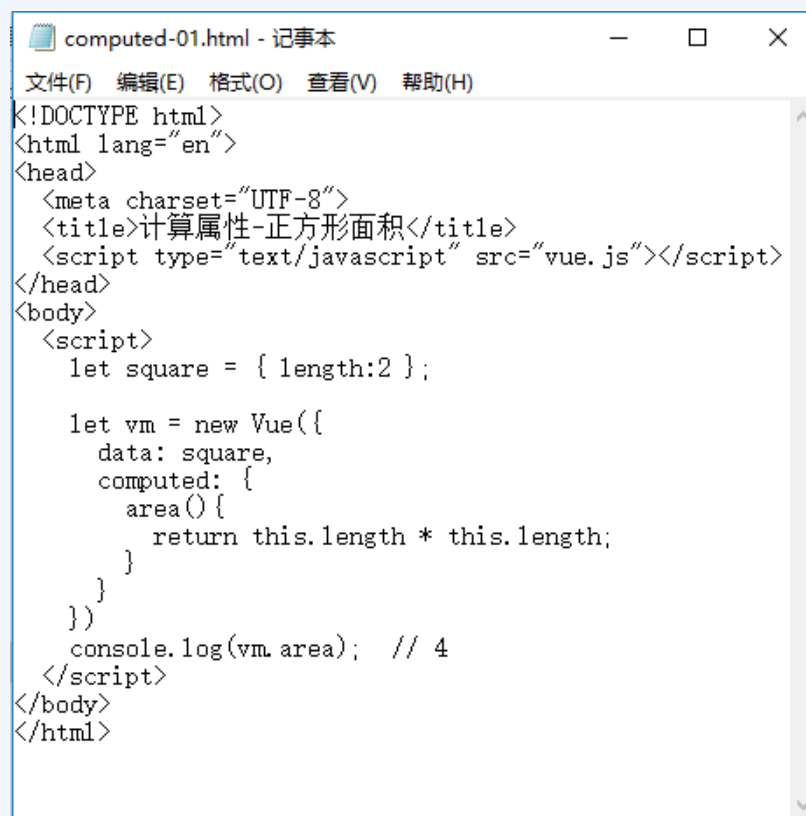
计算属性是“存取器”，本质上是函数，但在访问（调用）时需要向对待变量那样，不加小括号。

```
computed: {  
  //具体属性内容  
}
```

三. 计算属性与侦听器

❖ 计算属性

对象、属性、键值对



```
computed-01.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>计算属性-正方形面积</title>
  <script type="text/javascript" src="vue.js"></script>
</head>
<body>
  <script>
    let square = { length:2 };

    let vm = new Vue({
      data: square,
      computed: {
        area() {
          return this.length * this.length;
        }
      }
    })
    console.log(vm.area); // 4
  </script>
</body>
</html>
```

三. 计算属性与侦听器

❖ 计算属性

get、set 对象访问器



```
computed-02.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>计算属性-get和set方法</title>
  <script src="vue.js"></script>
</head>
<body>
  <script>
    let square = {length:2};

    let vm = new Vue({
      data: square,
      computed: {
        area: {
          get() {
            return this.length * this.length;
          },
          set(value) {
            this.length = Math.sqrt(value);
          }
        }
      }
    })

    console.log(vm.area); // 4
    vm.area = 9
    console.log(vm.length); // 3
  </script>
</body>
</html>
```

三. 计算属性与侦听器

❖ 计算属性与侦听器

一个应用程序中，变量之间存在很多关联关系。Vue.js提供了根据有些变量自动关联另一些变量的机制，以简化对象之间的复杂关系。

❖ 侦听器

在数据模型中的某个属性发生变化的时候进行拦截，从而执行指定的处理逻辑。其应用场景主要有两种：

1. **拦截操作：**当希望在某个属性发生变化的时候执行指定的操作。
2. **耗时操作：**对用户操作进行监听，只有超过多长时间才执行操作。

```
watch: {  
    //具体内容  
}
```


三. 计算属性与侦听器

❖ 侦听器

```
watch-01.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>侦听器-正方形面积</title>
  <script src="vue.js"></script>
</head>
<body>
  <script>
    let square = {
      length:2,
      area:4
    };

    let vm = new Vue({
      data: square,
      watch: {
        length(value) {
          this.area = value * value; //想像这个乘法计算是一个耗时较长的复杂运算
          console.log(vm.area); // 9
        }
      }
    })
    console.log(vm.area); // 4
    vm.length = 3
  </script>
</body>
</html>
```

普通用法

```
watch-03.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>侦听器-侦听对象内容变化成功</title>
  <script src="vue.js"></script>
</head>
<body>
  <script>
    let circle = {
      position: {
        x:0, y:0
      },
      radius: 10
    }
    let vm = new Vue({
      data: circle,
      watch: {
        position: {
          handler(newValue) {
            console.log(`在watch中侦听到，位置变化到了 (${newValue.x}, ${newValue.y})`);
          },
          deep: true
        }
      }
    })
    vm.position.x = 3;
  </script>
</body>
</html>
```

深度监听

```
watch-06.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>侦听器-immediate参数</title>
  <script src="vue.js"></script>
</head>
<body>
  <script>
    let circle = {
      position: {
        x:0, y:0
      },
      position2: {
        x:0, y:0
      }
    }
    let vm = new Vue({
      data: circle,
      watch: {
        position: {
          handler(value) {
            this.position2.x = this.position.x + Math.random();
            this.position2.y = this.position.y + Math.random();
            console.log(this.position2.x, this.position2.y);
          },
          deep: true,
          immediate: true
        }
      }
    })
    vm.position.x = 3;
  </script>
</body>
</html>
```

初始化执行监听

三. 计算属性与侦听器

❖ 侦听器——对数组进行侦听

使用标准方法修改数组才能被侦听到

`push()` 尾部添加

`pop()` 尾部删除

`unshift()` 头部添加

`shift()` 头部删除

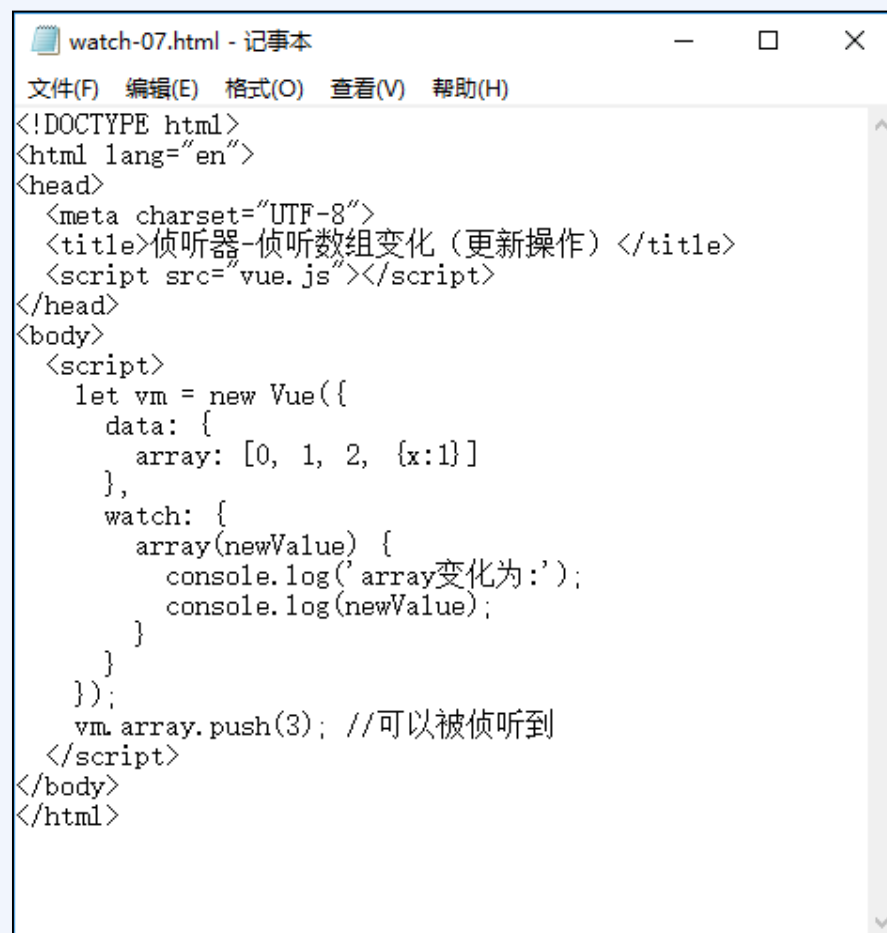
`splice()` 删除、添加、替换

`sort()` 排序

`reverse()` 逆序

三. 计算属性与侦听器

❖ 侦听器——对数组进行侦听



```
watch-07.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>侦听器-侦听数组变化(更新操作)</title>
  <script src="vue.js"></script>
</head>
<body>
  <script>
    let vm = new Vue({
      data: {
        array: [0, 1, 2, {x:1}]
      },
      watch: {
        array(newValue) {
          console.log('array变化为:');
          console.log(newValue);
        }
      }
    });
    vm.array.push(3); //可以被侦听到
  </script>
</body>
</html>
```

课程提纲

1

Web前端开发概述

2

Vue.js开发基础

3

计算属性与侦听器

4

事件处理

5

表单绑定

6

结构渲染

7

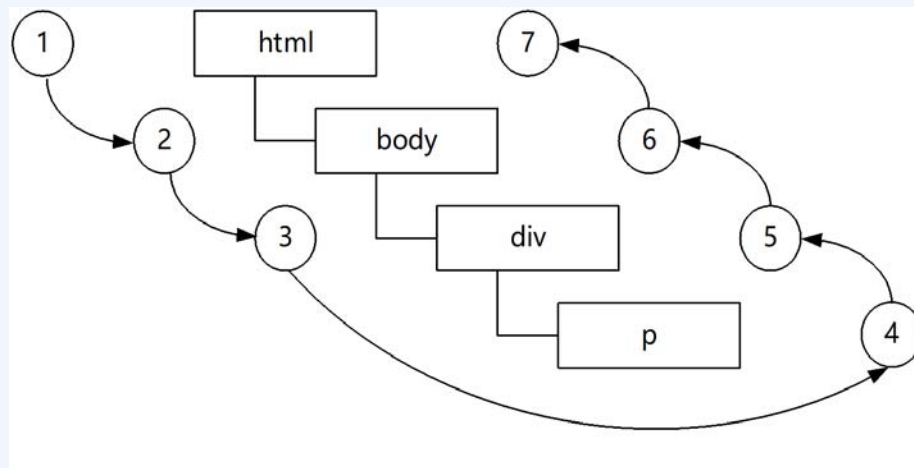
阶段案例

四. 事件处理

❖ 事件与事件流

事件是发生在HTML元素上的某些特定的事情，它的目的是使页面具有某些行为，执行某些“动作”，最终实现Web页面和用户的交互。在一个网页中，通常已经预先定义好了很多事件（例如一个页面完成加载、某个按钮被点击、将鼠标移动到某个元素上），开发人员可以编写相应的“事件处理程序”来响应相应的事件。

DOM（文档对象模型）是树状结构，事件流当某个子元素被点击时，它的父元素也被点击了……因此一次点击不是一个事件，而是一系列事件，从而构成“事件流”。



四. 事件处理

❖ 事件对象

浏览器支持的事件种类是非常多了，可以分为几类：

用户界面事件：涉及与BOM交互的通用浏览器事件。

焦点事件：在元素获得或者失去焦点时触发的事件。

鼠标事件：使用鼠标在页面上执行某些操作时触发的事件。

滚轮事件：使用鼠标滚轮时触发的事件。

输入事件：向文档中输入文本时触发的事件。

键盘事件：使用键盘在页面上执行某些操作时触发的事件。

输入法事件：使用某些输入法时触发的事件。

移动设备出现以后，又增加了“**触摸**”事件。

四. 事件处理

❖ 事件对象

标准DOM	类型	读/写	说明
altKey	Boolean	读写	按下ALT键则为true，否则为false
button	Integer	读写	鼠标事件，值对应按下的鼠标键，详见6.4.1节
cancelable	Boolean	只读	是否可以取消事件的默认行为
stopPropagation()	Function	N/A	可以调用该方法来阻止事件向上冒泡
clientX	Integer	只读	鼠标在客户端区域（当前窗口）的水平坐标，不包括工具栏、滚动条等
clientY	Integer	只读	鼠标在客户端区域（当前窗口）的垂直坐标，不包括工具栏、滚动条等
ctrlKey	Boolean	只读	按下Ctrl键则为true，否则为false
relatedTarget	Element	只读	鼠标所离开的元素
relatedTarget	Element	只读	鼠标正在进入的元素

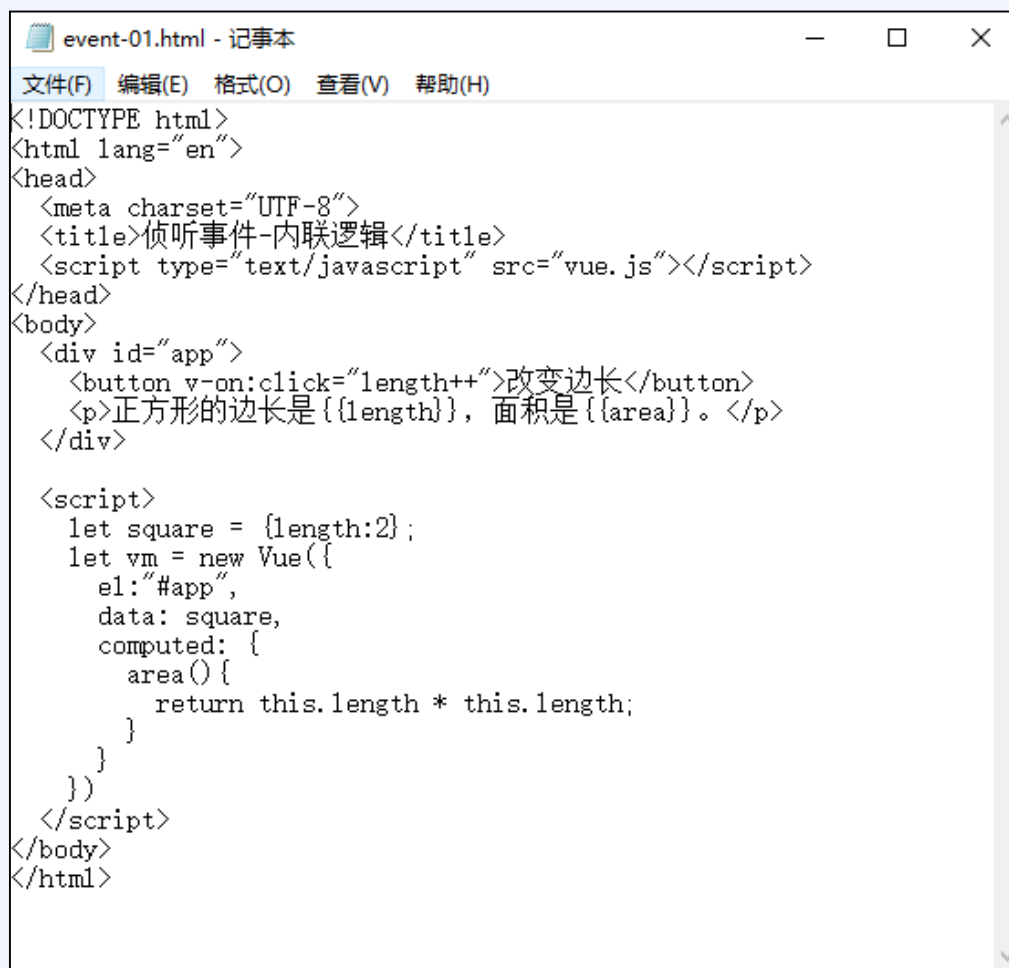
四. 事件处理

❖ 事件对象

标准DOM	类型	读/写	说明
charCode	Integer	只读	按下按键的Unicode值
keyCode	Integer	读写	keypress时为0，其余为按下按键的数字代号。
detail	Integer	只读	鼠标按钮点击的次数
preventDefault()	Function	N/A	可以调用该方法来阻止事件的默认行为
screenX	Integer	只读	鼠标相对于屏幕的水平坐标
screenY	Integer	只读	鼠标相对于屏幕的垂直坐标
shiftKey	Boolean	只读	按下Shift键则为true，否则为false
target	Element	只读	引起事件的元素/对象
type	String	只读	事件的名称

四. 事件处理

❖ 以内联方式响应事件



```
event-01.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>侦听事件-内联逻辑</title>
  <script type="text/javascript" src="vue.js"></script>
</head>
<body>
  <div id="app">
    <button v-on:click="length++">改变边长</button>
    <p>正方形的边长是{{length}}, 面积是{{area}}。</p>
  </div>

  <script>
    let square = {length:2};
    let vm = new Vue({
      el: "#app",
      data: square,
      computed: {
        area() {
          return this.length * this.length;
        }
      }
    })
  </script>
</body>
</html>
```

四. 事件处理

❖ 事件处理方法

```
event-02.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>侦听事件-方法内逻辑</title>
  <script type="text/javascript" src="vue.js"></script>
</head>
<body>
  <div id="app">
    <button v-on:click="changeLength">改变边长</button>
    <p>正方形的边长是{{length}}, 面积是{{area}}。</p>
  </div>

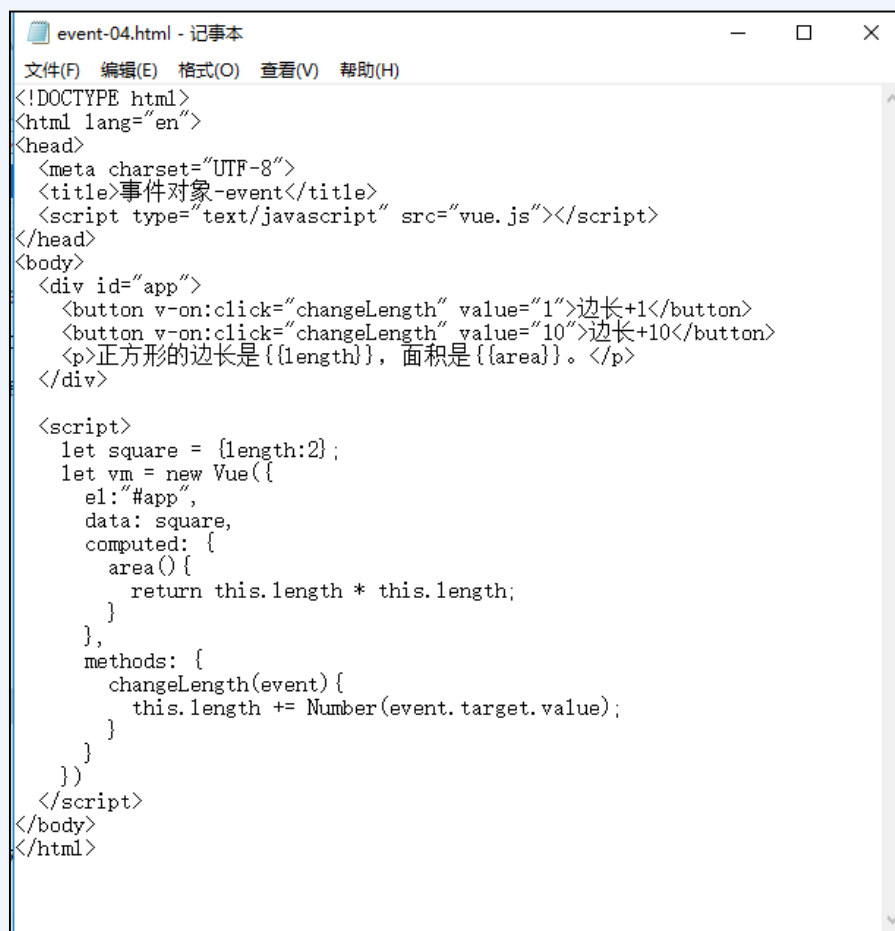
  <script>
    let square = {length:2};
    let vm = new Vue({
      el:"#app",
      data: square,
      computed: {
        area() {
          return this.length * this.length;
        }
      },
      methods: {
        changeLength() {
          this.length++;
        }
      }
    })
  </script>
</body>
</html>
```

```
event-03.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>侦听事件-方法内传参</title>
  <script type="text/javascript" src="vue.js"></script>
</head>
<body>
  <div id="app">
    <button v-on:click="changeLength(1)">边长+1</button>
    <button v-on:click="changeLength(10)">边长+10</button>
    <p>正方形的边长是{{length}}, 面积是{{area}}。</p>
  </div>

  <script>
    let square = {length:2};
    let vm = new Vue({
      el:"#app",
      data: square,
      computed: {
        area() {
          return this.length * this.length;
        }
      },
      methods: {
        changeLength(delta) {
          this.length += delta;
        }
      }
    })
  </script>
</body>
</html>
```

四. 事件处理

❖ 使用事件对象



```
event-04.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>事件对象-event</title>
  <script type="text/javascript" src="vue.js"></script>
</head>
<body>
  <div id="app">
    <button v-on:click="changeLength" value="1">边长+1</button>
    <button v-on:click="changeLength" value="10">边长+10</button>
    <p>正方形的边长是 {{length}}, 面积是 {{area}}。</p>
  </div>

  <script>
    let square = {length:2};
    let vm = new Vue({
      el: "#app",
      data: square,
      computed: {
        area() {
          return this.length * this.length;
        }
      },
      methods: {
        changeLength(event) {
          this.length += Number(event.target.value);
        }
      }
    })
  </script>
</body>
</html>
```

四. 事件处理

❖ 动手练习：监视鼠标移动



```
event-06.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>监视鼠标移动</title>
  <script type="text/javascript" src="vue.js"></script>
  <style>
    #app{width: 128px; height: 128px;}
  </style>
</head>
<body>
  <div id="app">
    <p>鼠标位于({{x}}, {{y}})</p>
    <p>背景色 {{backgroundColor}}</p>
    <div v-on:mousemove="mouseMove" v-bind:style="{backgroundColor}">
    </div>
  </div>
</body>
</html>

<script>
let vm = new Vue({
  el: "#app",
  data: {x:0, y:0},
  methods: {
    mouseMove(event) {
      this.x = event.offsetX;
      this.y = event.offsetY;
    }
  },
  computed: {
    backgroundColor() {
      const c = (this.x+this.y).toString(16);
      return c.length==2 ? `#${c}${c}${c}` : `#0${c}0${c}0${c}`;
    }
  }
})
</script>
```

四. 事件处理

❖ 事件修饰符

Vue.js为v-on指令提供了“事件修饰符”，通过它们可以用声明的方式实现对事件的附加修饰/限制。

.stop，阻止事件的
“冒泡”传播。

```
event-07.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>事件冒泡</title>
  <script type="text/javascript" src="vue.js"></script>
</head>
<body>
  <div id="app">
    <div id="outer" v-on:click="show('外层div被点击');">
      <div id="inner" v-on:click="show('内层div被点击');"></div>
    </div>
  </div>
</body>
</html>

<script>
let vm = new Vue({
  el: '#app',
  methods: {
    show(message) {
      alert(message);
    }
  }
})
</script>
</body>
</html>
```

```
event-08.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>修饰符.stop</title>
  <script type="text/javascript" src="vue.js"></script>
</head>
<body>
  <div id="app">
    <div id="outer" v-on:click="show('外层div被点击');">
      <div id="inner" v-on:click.stop="show('内层div被点击');"></div>
    </div>
  </div>
</body>
</html>

<script>
let vm = new Vue({
  el: '#app',
  methods: {
    show(message) {
      alert(message);
    }
  }
})
</script>
</body>
</html>
```

四. 事件处理

❖ 事件修饰符

.self, .capture, .once, .prevent

```
event-09.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>修饰符.self</title>
  <script type="text/javascript" src="vue.js"></script>
  <style>
    div#app div{
      border:2px solid #555
    }
    #outer{
      width:200px;
      padding:30px;
    }
    #inner{
      height:30px;
    }
  </style>
</head>
<body>
  <div id="app">
    <div id="outer" v-on:click.self="show('外层div被点击');">
      <div id="inner" v-on:click="show('内层div被点击');"></div>
    </div>
  </div>

  <script>
    let vm = new Vue({
      el:"#app",
      methods:{
        show(message){
          alert(message);
        }
      }
    })
  </script>
</body>
</html>
```

```
event-10.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>修饰符.capture</title>
  <script type="text/javascript" src="vue.js"></script>
  <style>
    div#app div{
      border:2px solid #555
    }
    #outer{
      width:200px;
      padding:30px;
    }
    #inner{
      height:30px;
    }
  </style>
</head>
<body>
  <div id="app">
    <div id="outer" v-on:click.capture="show('外层div被点击');">
      <div id="inner" v-on:click="show('内层div被点击');"></div>
    </div>
  </div>

  <script>
    let vm = new Vue({
      el:"#app",
      methods:{
        show(message){
          alert(message);
        }
      }
    })
  </script>
</body>
</html>
```

四. 事件处理

❖ 事件修饰符

.self, .capture, .once, .prevent

```
event-11.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>修饰符.once</title>
  <script type="text/javascript" src="vue.js"></script>
  <style>
    div#app div{
      border:2px solid #555
    }
    #outer{
      width:200px;
      padding:30px;
    }
    #inner{
      height:30px;
    }
  </style>
</head>
<body>
  <div id="app">
    <div id="outer" v-on:click.once="show('外层div被点击);">
      <div id="inner"></div>
    </div>
  </div>

  <script>
    let vm = new Vue({
      el:"#app",
      methods:{
        show(message){
          alert(message);
        }
      }
    })
  </script>
</body>
</html>
```

```
event-12.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>修饰符.prevent</title>
  <script type="text/javascript" src="vue.js"></script>
  <style>
    div#app div{
      border:2px solid #555
    }
    #outer{
      width:200px;
      padding:30px;
    }
    #inner{
      height:30px;
    }
  </style>
</head>
<body>
  <div id="app">
    <div id="outer">
      <div id="inner">
        <!-- <a href="http://www.artech.cn" v-on:click="show('链接被点击)'" -->
          这是一个链接
        </a> -->
        <a href="http://www.artech.cn" v-on:click.prevent="show('链接被点击)'" -->
          这是一个链接
        </a>
      </div>
    </div>
  </div>

  <script>
    let vm = new Vue({
      el:"#app",
      methods:{
        show(message){

```

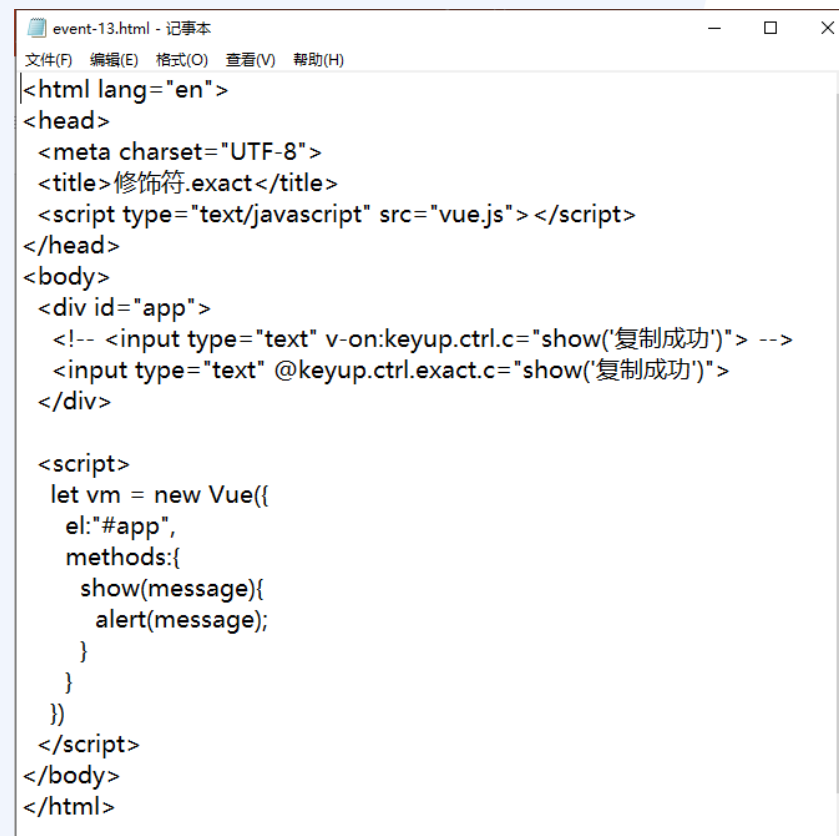
四. 事件处理

❖ 其他修饰符

按键修饰符: `keydown`、`keypress`、`keyup`

系统修饰符: `.ctrl`、`.alt`、`.shift`、`.meta`

鼠标按钮修饰符: `.left`、`.right`、`.middle`



```
event-13.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>修饰符.exact</title>
  <script type="text/javascript" src="vue.js"></script>
</head>
<body>
  <div id="app">
    <!-- <input type="text" v-on:keyup.ctrl.c="show('复制成功')"> -->
    <input type="text" @keyup.ctrl.exact.c="show('复制成功')">
  </div>

  <script>
    let vm = new Vue({
      el:"#app",
      methods:{
        show(message){
          alert(message);
        }
      }
    })
  </script>
</body>
</html>
```


课程提纲

1

Web前端开发概述

2

Vue.js开发基础

3

计算属性与侦听器

4

事件处理

5

表单绑定

6

结构渲染

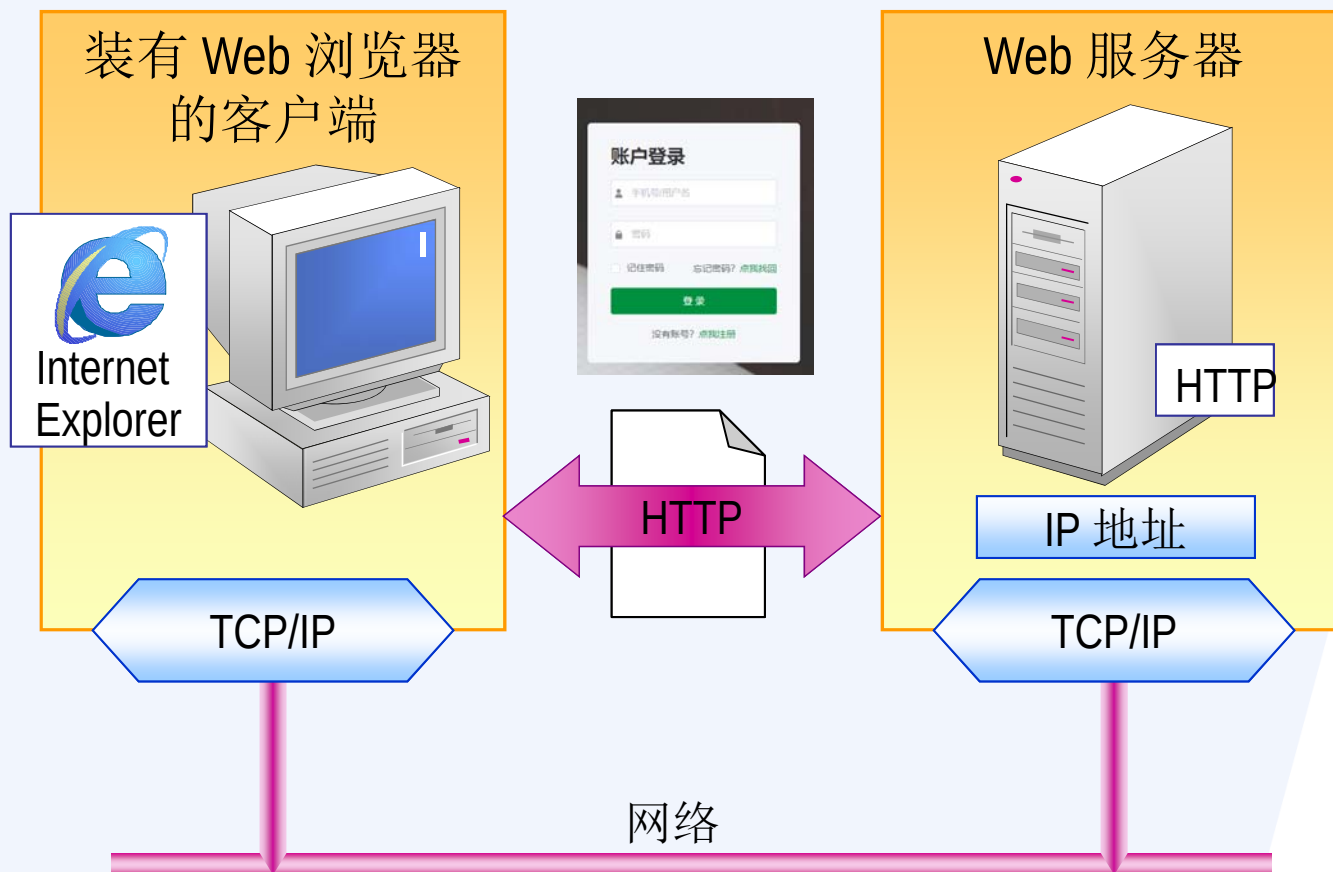
7

阶段案例

五. 表单绑定

❖ 什么是表单

表单用于实现浏览器Web端和服务器的各种逻辑交互，使用v-model指令进行绑定。



五. 表单绑定

❖ 文本框（单行/多行）

```
form-01.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>表单绑定-文本框text（常用写法）</title>
  <script type="text/javascript" src="vue.js"></script>
</head>
<body>
  <div id="app">
    <input v-model="name" placeholder="请输入姓名">
    <p>{{ name }} 您好! </p>
  </div>
  <script>
    let vm = new Vue({
      el: "#app",
      data: {
        name: ""
      }
    })
  </script>
</body>
</html>
```

```
form-02.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>表单绑定-文本框text（v-model的原理）</title>
  <script type="text/javascript" src="vue.js"></script>
</head>
<body>
  <div id="app">
    <input v-bind:value="name" v-on:input="name =
    $event.target.value" placeholder="请输入姓名">
    <p>{{ name }} 您好! </p>
  </div>
  <script>
    let vm = new Vue({
      el: "#app",
      data: {
        name: ""
      }
    })
  </script>
</body>
</html>
```

```
form-03.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>表单绑定-多文本框textarea</title>
  <script type="text/javascript" src="vue.js"></script>
</head>
<body>
  <div id="app">
    <textarea rows="4" v-model="comment"
    placeholder="请输入您的留言"></textarea>
    <p>您的留言是</p>
    <p style="white-space: pre-line;">{{ comment
  </p>
  </div>
  <script>
    let vm = new Vue({
      el: "#app",
      data: {
        comment: ""
      }
    })
  </script>
</body>
</html>
```

五. 表单绑定

❖ 单选框/复选框

```
form-04.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>表单绑定-单选按钮radio</title>
  <script type="text/javascript" src="vue.js"></script>
</head>
<body>
<div id="app">
  <span>选择一种语言: {{ language }}</span>
  <br/>
  <input type="radio" id="python"
    value="Python" v-model="language" />
  <label for="python">Python</label>

  <input type="radio" id="javascript"
    value="JavaScript" v-model="language" />
  <label for="javascript">JavaScript</label>

  <input type="radio" id="pascal"
    value="Pascal" v-model="language" />
  <label for="pascal">Pascal</label>
</div>
<script>
  let vm = new Vue({
    el: "#app",
    data: {
      language: ""
    }
  })
</script>
</body>
</html>
```

```
form-05.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>表单绑定-复选框checkbox</title>
  <script type="text/javascript" src="vue.js"></script>
</head>
<body>
  <div id="app">
    <span>请选择语言, 已选择 {{ languages.length }}</span>
  </div>
</body>
</html>

form-07.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>表单绑定-复选框checkbox</title>
  <script type="text/javascript" src="vue.js"></script>
</head>
<body>
  <div id="app">
    <input type="checkbox" id="agree" v-model="agree"
      true-value="同意" false-value="不同意" />
    <label for="agree">{{ agree }}</label>
  </div>
<script>
  let vm = new Vue({
    el: "#app",
    data: {
      agree: '不同意'
    }
  })
</script>
</body>
</html>
```

```
form-06.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>表单绑定-复选框checkbox</title>
  <script type="text/javascript" src="vue.js"></script>
</head>
<body>
  <div id="app">
    <input type="checkbox" id="agree" v-model="agree" />
    <label for="agree">{{ agree ? "同意" : "不同意" }}</label>
  </div>
  <script>
    let vm = new Vue({
      el: "#app",
      data: {
        agree: false
      }
    })
  </script>
</body>
</html>
```

五. 表单绑定

❖ 下拉框/多选列表框

```
form-08.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>表单绑定-下拉框select</title>
  <script type="text/javascript" src="vue.js"></script>
</head>
<body>
  <div id="app">
    <span>选择一种语言: {{ language }}</span>
    <br/>
    <select v-model="language">
      <option disabled value="">
      <option>python</option>
      <option>JavaScript</option>
      <option>Pascal</option>
    </select>
  </div>
  <script>
    let vm = new Vue({
      el: "#app",
      data: {
        language: ""
      }
    })
  </script>
</body>
</html>

form-09.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>表单绑定-多选列表框select</title>
  <script type="text/javascript" src="vue.js"></script>
</head>
<body>
  <div id="app">
    <span>请选择语言, 已选择 {{ selected.length }} 种</span>
    <br/>
    <select multiple v-model="selected">
      <option v-for="option in languages" v-bind:value="option.value">
        {{option.text}}
      </option>
    </select>
  </div>
  <script>
    let vm = new Vue({
      el: "#app",
      data: {
        selected: [],
        languages: [
          {text: 'Python', value: 101},
          {text: 'JavaScript', value: 102},
          {text: 'Pascal', value: 103}
        ]
      }
    })
  </script>
</body>
</html>
```

五. 表单绑定

❖ 表单绑定修饰符

.lazy, .number, .trim

```
form-11.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>表单绑定-修饰符.lazy</title>
  <script type="text/javascript" src="vue.js"></script>
</head>
<body>
  <div id="app">
    <input v-model.lazy="msg">
    <span>{{msg}}</span>
  </div>
  <script>
    let vm = new Vue({
      el: "#app",
      data: {
        msg: '111',
      }
    })
  </script>
</body>
</html>
```

```
form-12.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>表单绑定-修饰符.number</title>
  <script type="text/javascript" src="vue.js"></script>
</head>
<body>
  <div id="app">
    <input v-model.number="age" type="number">
    <span>{{age}}</span>
  </div>
  <script>
    let vm = new Vue({
      el: "#app",
      data: {
        age: '',
      },
      watch: {
        age() {
          console.log(typeof(this.age))
        }
      }
    })
  </script>
</body>
</html>
```

```
form-13.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>表单绑定-修饰符.trim</title>
  <script type="text/javascript" src="vue.js"></script>
</head>
<body>
  <div id="app">
    <input v-model.trim="msg">
    <span>一共有{{msg.length}}个字符</span>
  </div>
  <script>
    let vm = new Vue({
      el: "#app",
      data: {
        msg: '',
      }
    })
  </script>
</body>
</html>
```

课程提纲

1

Web前端开发概述

2

Vue.js开发基础

3

计算属性与侦听器

4

事件处理

5

表单绑定

6

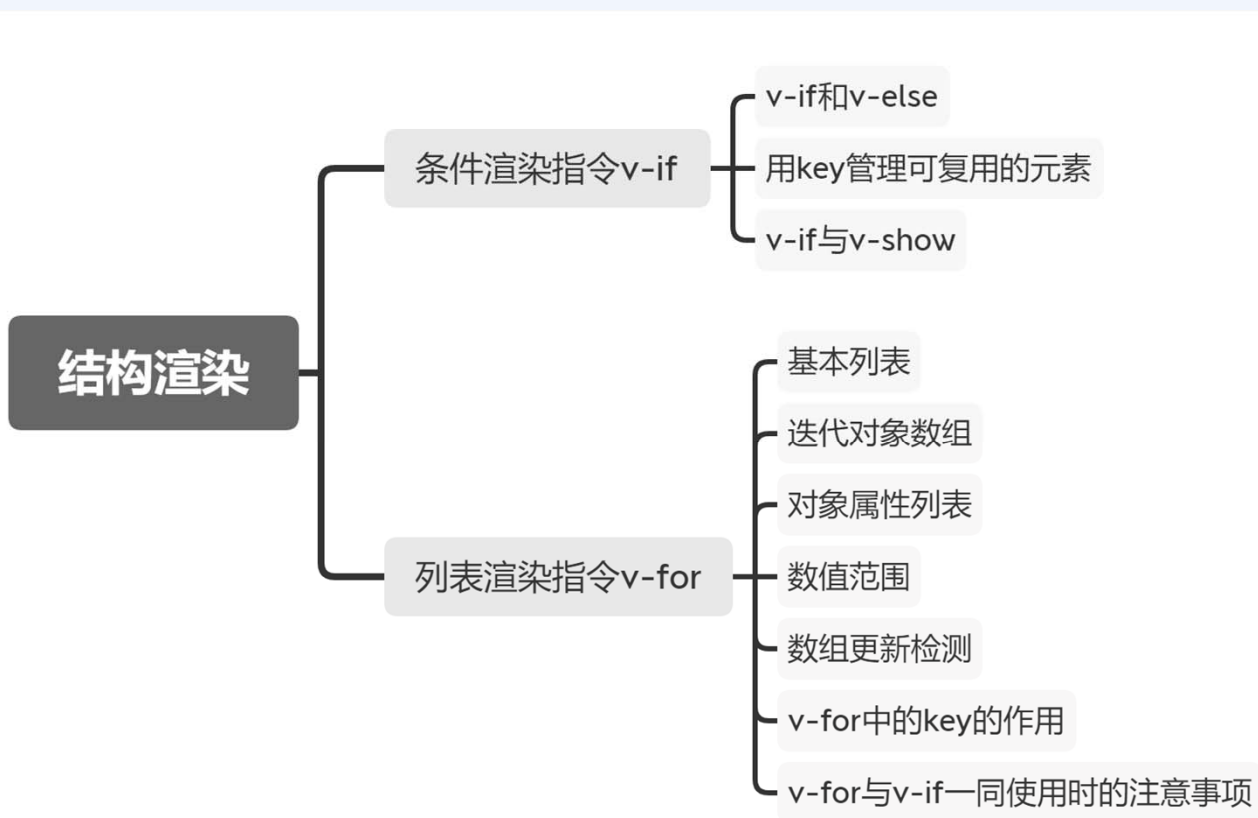
结构渲染

7

阶段案例

六. 结构渲染

主要包括条件渲染（v-if、v-else）和列表渲染（v-for），即根据一定的条件或者按照一定的规则生成DOM元素。



六. 结构渲染

❖ 条件渲染指令（v-if、v-else）

```
v-if-v-for-01.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>v-if和v-else</title>
  <script type="text/javascript" src="vue.js"></script>
  <style>
    button {
      margin-top: 20px;
    }
  </style>
</head>
<body>
  <div id="app">
    <div v-if="loginByMobile">
      <label>手机号登录</label>
      <input type="text" placeholder="请输入手机号">
    </div>
    <div v-else>
      <label>邮箱登录</label>
      <input type="text" placeholder="请输入邮箱">
    </div>
    <button v-on:click="loginByMobile = !loginByMobile">更换登录方式</button>
  </div>

  <script>
    var vm = new Vue({
      el: '#app',
      data: {
        loginByMobile: true
      }
    })
  </script>
</body>
</html>
```

```
v-if-v-for-02.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>v-else-if</title>
  <script type="text/javascript" src="vue.js"></script>
  <style>
    button {
      margin-top: 20px;
    }
  </style>
</head>
<body>
  <div id="app">
    <div v-if="type === 'A'">
      A区域
    </div>
    <div v-else-if="type === 'B'">
      B区域
    </div>
    <div v-else-if="type === 'C'">
      C区域
    </div>
    <div v-else>
      A/B/C都不是
    </div>
  </div>

  <script>
    var vm = new Vue({
      el: '#app',
      data: {
        type: 'B'
      }
    })
  </script>
</body>
</html>
```

六. 结构渲染

❖ 条件渲染指令（v-if、v-show、v-for）

```
v-if-v-for-04.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>v-if和v-show</title>
  <script type="text/javascript" src="vue.js"></script>
  <style>
    button {
      margin-top: 20px;
    }
  </style>
</head>
<body>
  <div id="app">
    <div v-show="loginByMobile">
      <label>手机号登录</label>
      <input type="text" placeholder="请输入手机号"
key="by-mobile">
    </div>
    <div v-show="!loginByMobile">
      <label>邮箱登录</label>
      <input type="text" placeholder="请输入邮箱"
key="by-email">
    </div>
    <button v-on:click="loginByMobile = !
loginByMobile">更换登录方式</button>
  </div>

  <script>
    var vm = new Vue({
      el: '#app',
      data: {
        loginByMobile: true
      }
    })
  </script>
</body>
</html>
```

```
v-if-v-for-05.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>v-for迭代普通数组</title>
  <script type="text/javascript" src="vue.js"></script>
</head>
<body>
  <div id="app">
    <ul>
      <li v-for="item in list">{{item}}</li>
    </ul>
  </div>

  <script>
    var vm = new Vue({
      el: '#app',
      data: {
        list: ['阳光', '空气', '沙滩', '草地']
      }
    })
  </script>
</body>
</html>
```

课程提纲

1

Web前端开发概述

2

Vue.js开发基础

3

计算属性与侦听器

4

事件处理

5

表单绑定

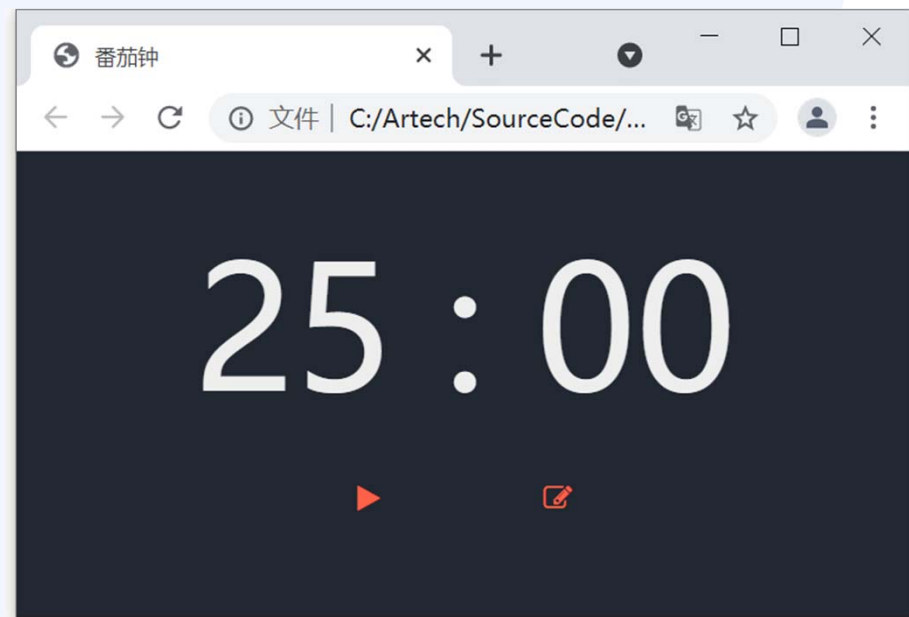
6

结构渲染

7

阶段案例

七. 阶段案例



The background features a central horizontal band with a blue gradient. Within this band, there are several glowing, metallic-looking spheres of varying sizes. Some spheres are positioned to the left and right, while a cluster of smaller spheres forms a bright, multi-pointed starburst in the center. Thin, white, horizontal lines radiate from the central starburst, extending towards the left and right edges of the blue band. The overall design is modern and high-tech.

Thank You !